Backend Endpoints for AI Item Identification

Cadra Mobile · Barcode Station · What the mobile app needs from the backend team · Oct 13, 2026

Base URL: `https://staging.app.cadra.ng/api/v1`. All paths below are relative to it. Six existing endpoints are reused as they are and 18 are new. Existing endpoints were checked against the live staging OpenAPI spec on Oct 13, 2026.

1. Already exists — reuse as is

Endpoint	Purpose
`GET /asset-categories`	Class catalog, cached offline
`GET /locations (+ /{id}/children)`	Rooms and zones, cached offline
`GET /assets/by-tag/{tag}`	Audit verification, lost-tag confirm
`POST /assets/{id}/tag-affixed`	Confirm the printed tag was put on the item
`POST /assets/{id}/verify`	Audit sighting
`POST /assets/{id}/mark-missing, /mark-found`	Lost-tag flow

2. New — needed first (offline core and codes)

Endpoint	Purpose
`POST /devices`	Register a device, get a `device_id`
`POST /devices/{id}/code-blocks`	Reserve a block of asset codes for offline use
`GET /devices/{id}/code-blocks`	Check balance and status of reserved blocks
`POST /devices/{id}/code-blocks/{block_id}/return`	Return unused codes on sync
`GET /ai/config`	Thresholds: confidence, duplicate similarity, capitalisation value, code block size

3. New — registering items

Endpoint	Purpose
`POST /ai-captures/photos`	Upload one photo (multipart, idempotent by `photo_id`)
`POST /ai-captures`	Register an item with its pre-issued code, confirmed values, AI suggestions, changed fields, room context, photo IDs and signature. Idempotent by `capture_id`
`GET /ai-captures/{capture_id}`	Check sync status of a capture
`POST /ai-captures/{capture_id}/void`	Void an issued code with a reason (codes are never reused)
`GET /asset-classes/{id}/required-fields`	Mandatory attributes per class, or add them to `/asset-categories`

4. New — duplicates, lost tags, audits

Endpoint	Purpose
`GET /ai/signatures?location_id=&since=`	Download signatures for rooms, as changes since the last sync
`GET /assets?location_id=&updated_since=`	Asset summaries for offline lookup (could extend the existing `GET /assets`)
`GET /assets/{id}/photos`	Stored reference photos to show beside a possible duplicate or match
`POST /ai/lost-tag-requests`	Ask custodian and supervisor to approve a reprint when items are identical
`POST /assets/{id}/swap-flags`	Flag a possible tag swap found during an audit
`GET /ai-captures/flags?since=`	Duplicates the server found across devices after sync

5. New — printing and models

Endpoint	Purpose
`GET /barcode/label-template`	Label layout and code format, so the phone can print offline
`GET /ai/models/manifest + file download`	Model versions with checksums. Without an ML team this can be a simple file host

The most important rule. `POST /ai-captures` must accept a code the phone already issued, and must never create a second asset when the same `capture_id` is sent again.

6. Working without an ML team

The addendum assumes models trained on NRS's furniture catalog, plus the Galadriel server for re-checking captures and retraining. Without anyone to do that:

- **Server re-checking and retraining (AI-FR-14) drop out.** The backend can still store operator corrections for later, but nothing consumes them.
- **The visual signature still works** with a ready-made image model, so duplicate checks, lost-tag recovery and audit verification are fine.
- **Class suggestions are the weak point.** Generic models don't know NRS sub-types like "executive chair". A workable approach without training is to suggest the class of the most similar items operators have already confirmed. It starts weak and improves as the register fills, needs no ML team, and uses the same signatures.
- **Room counting needs a detector.** Off-the-shelf detectors recognise chairs, tables, sofas and some other common furniture. Cabinets, shelving and partitions would need training.